

PRÁCTICA 2 — METAHEURÍSTICAS

Optimización del Problema de Asignación Cuadrática (QAP) en la Planta Ford Valencia



Autores

Alumno 1: José Javier Torres Cantero

Correo: jjtc0005@red.ujaen.es

Alumno 2: David Fernández Espejo

Correo: dfe00004@red.ujaen.es

Grado en Ingeniería Informática

Asignatura: Metaheurísticas

Curso: 2025–2026

ÍNDICE

1. Algoritmos Evolutivos

- 1.1. Parámetros del programa
- 1.2. Explicación general de ejecución
 - 1.2.1. Inicialización de la población
 - 1.2.2. Evaluación de la población
 - 1.2.3. Selección de padres
 - 1.2.4. Generación de descendientes
 - 1.2.5. Reemplazo y actualización de la población
 - 1.2.6. Criterio de parada
- 1.3. Estacionario
 - 1.3.1. Funcionamiento
 - 1.3.2. Estructuras y variables utilizadas
 - 1.3.3. Pseudocódigo
- 1.4. Evolutivo
 - 1.4.1. Funcionamiento
 - 1.4.2. Estructuras y variables utilizadas
 - 1.4.3. Pseudocódigo

2. Operadores

- 2.1. MOC
 - 2.1.1. Definición
 - 2.1.2. Funcionamiento
 - 2.1.3. Pseudocódigo
- 2.2. OX2
 - 2.2.1. Definición
 - 2.2.2. Funcionamiento
 - 2.2.3. Pseudocódigo
- 2.3. Mutación
 - 2.3.1. Definición
 - 2.3.2. Funcionamiento
 - 2.3.3. Pseudocódigo

3. Resultados

- 3.1. Generacional OX2
 - 3.1.1. Calidad de las soluciones obtenidas
 - 3.1.2. Estabilidad y variabilidad
 - 3.1.3. Influencia del parámetro kBest
 - 3.1.4. Influencia del elitismo

3.1.5. Tiempo de ejecución

3.2. Generacional MOC

3.2.1. Calidad de las soluciones obtenidas

3.2.2. Estabilidad y variabilidad

3.2.3. Influencia del parámetro kBest

3.2.4. Influencia del elitismo

3.2.5. Tiempo de ejecución

3.3. Generacional OX2 / MOC

3.4. Estacionario OX2 / MOC

3.5. Comparación general (Mejor Generacional, Estacionario, Pr1)

4. Logs

4.1. Acceso a los logs de ejecución

1. Algoritmos Evolutivos

Los **algoritmos evolutivos** constituyen una familia de métodos de optimización inspirados en los principios fundamentales de la selección natural. Su objetivo es resolver problemas complejos mediante la simulación de procesos biológicos como la reproducción, la variación genética y la supervivencia del más apto. A diferencia de los métodos deterministas tradicionales, estos algoritmos operan sobre un conjunto de soluciones simultáneamente, lo que les permite explorar el espacio de búsqueda de manera diversa y paralela.

En esencia, un algoritmo evolutivo mantiene una **población de individuos**, donde cada individuo representa una solución candidata al problema. A lo largo de sucesivas iteraciones —denominadas **generaciones**— la población se transforma aplicando tres operadores fundamentales:

- Selección: que favorece a los individuos con mejor desempeño;
- Cruce: encargado de recombinar información genética procedente de varios padres;
- Mutación: que introduce pequeñas alteraciones aleatorias para preservar la diversidad.

Este proceso iterativo permite que las soluciones evolucionen progresivamente hacia configuraciones de menor coste o mayor calidad, dependiendo del criterio del problema. Además, el carácter estocástico del método proporciona la capacidad de escapar de óptimos locales, una ventaja especialmente valiosa en problemas combinatorios difíciles como el **Quadratic Assignment Problem (QAP)**, que presenta un espacio de búsqueda altamente multimodal y de enorme dimensionalidad.

Los algoritmos evolutivos destacan precisamente en este tipo de entornos porque combinan **exploración global** y **explotación local** de forma equilibrada. Gracias a su flexibilidad, pueden adaptarse a diversas variantes: algoritmos genéticos generacionales, modelos estacionarios, enfoques híbridos o estrategias evolutivas, entre otros. Cada configuración introduce matices distintos en la dinámica poblacional, ajustando la presión selectiva, la velocidad de convergencia y la capacidad para mantener diversidad genética.

1.1 Parámetros del programa

En la práctica anterior ya definimos todos los parámetros básicos que ejecutan el programa, como el archivo con la ruta al fichero de los datos, el algoritmo a utilizar o las semillas.

Para esta práctica solo nos centraremos en todos los nuevos parámetros:

- **tamPoblacion**: es un entero que indica el número total de los individuos de una población.
 - **evaluaciones**: es un entero que indica cual es el límite total de evaluaciones del algoritmo.
 - **operadorCruce**: es un string que indicará el operador de cruce que se va a utilizar en la ejecución del algoritmo.
 - **elitismo**: es un entero que indica el número de individuos de una población que no se pierden entre generaciones, es decir, los n mejores globales de la generación actual se copian directamente en la generación siguiente.
 - **pobGreedy**: Es un flotante que indica el porcentaje de la población inicial que se genera mediante un algoritmo GreedyRandom.
 - **probCruce**: es un flotante que indica la probabilidad con la que se aplica el cruce. No en todas las reproducciones se combina el material genético de los padres cuando no hay cruce, el hijo es esencialmente una copia del progenitor seleccionado.
 - **probMutacion**: es un flotante que indica la probabilidad de aplicar el operador de mutación definido previamente sobre cada hijo.
 - **kBest**: es un entero que indicará el número de los mejores individuos de un torneo.
 - **kWorst**: es un entero que indicará el número de los peores individuos de un torneo.
-

1.2 Explicación general de ejecución

Los algoritmos evolutivos implementados en esta práctica siguen una misma estructura lógica compuesta por cinco fases: inicialización, evaluación, selección, generación de descendencia y reemplazo. Aunque existen variantes (generacional y estacionaria), ambos comparten el mismo esquema conceptual. El flujo general de ejecución es el siguiente:

1.2.1 Inicialización de la población

Se genera una población de tamaño fijo compuesta por distintas soluciones del problema. Una proporción **probGreedy** se construye mediante un método **GreedyRandom** para garantizar un nivel inicial de calidad, mientras que el resto se genera aleatoriamente para mantener la diversidad.

1.2.2 Evaluación de la población

Cada individuo se evalúa mediante la función de coste del **QAP**, calculando el valor objetivo asociado a su permutación.

Tras evaluar a todos, se identifica el mejor individuo inicial, que se almacenará como referencia global durante toda la evolución.

1.2.3 Selección de padres

En cada iteración (o en cada creación de hijos), se seleccionan los progenitores utilizando **torneos de tamaño kBest**.

Este método compara un número reducido de candidatos y elige siempre el de menor coste, introduciendo presión selectiva pero manteniendo una probabilidad razonable de explorar otras zonas de búsqueda.

1.2.4 Generación de descendencia

A partir de los padres seleccionados, se generan uno o dos hijos mediante los operadores evolutivos:

- **Cruce OX2 o MOC**, que combinan información de los progenitores manteniendo la validez de la permutación.
- **Mutación 2-opt**, aplicada con probabilidad **probMutacion**, para introducir variaciones locales que permitan escapar de óptimos locales.

En el modelo generacional (**GEN**), este proceso se repite hasta generar una población completa de nuevos individuos.

En el modelo estacionario (**EST**), sólo se generan dos hijos por iteración.

1.2.5 Reemplazo y actualización de la población

Dependiendo del modelo:

- En el **generacional**, la nueva población sustituye por completo a la anterior, incorporando además un número fijo de individuos élite (**elitismo**) que se mantienen sin modificaciones.
- En el **estacionario**, los dos hijos producidos reemplazan únicamente a los peores individuos, identificados mediante torneos de tamaño **kWorst**.

Tras el reemplazo, se actualiza el mejor individuo global si uno de los nuevos hijos mejora la mejor solución encontrada hasta el momento.

1.2.6 Criterio de parada:

El proceso evolutivo continúa mientras no se alcance el número máximo de evaluaciones (**evaluaciones**).

Una vez alcanzado el límite, el algoritmo se detiene y devuelve el mejor individuo obtenido.

1.3 Algoritmo Generacional

Es un tipo de algoritmo genético donde la idea clave es crear una población completamente nueva en cada generación.

1.3.1 Funcionamiento

- **Número de descendientes:** genera **tantos hijos como tamaño de población** (100 por defecto).
 - **Reemplazo completo:** la población antigua se sustituye totalmente por la nueva salvo por el elitismo.
 - **Elitismo explícito:** copia directamente los **elitismo = 2** mejores individuos en la nueva población.
 - **Cruce probabilístico:** se aplica el cruce entre los individuos actuales para obtener una nueva generación.
 - **Mutación:** aplicada a **hijo1 e hijo2** (pero solo **hijo1** se inserta en población), para obtener cambios entre el padre y los hijos y obtener un nuevo individuo.
 - **Evolución más agresiva:** grandes saltos entre generaciones y pérdida de diversidad más rápida.
-

1.3.2 Estructuras y variables utilizadas:

Recordamos algunas de las estructuras que utilizamos en la práctica anterior como:

- **Instance inst:** representación del problema con las matrices de flujos y distancias que cargaremos previamente gracias a la clase ConfigLoader.
- **int solucionInicial []:** vector de enteros que representa la solución inicial a modificar.
- **string semillas []:** vector de strings que representa las semillas a utilizar con nuestros **DNIs**.
- **int posSemilla:** entero que indica la semilla a utilizar del vector de semillas.

- **int evaluaciones:** entero que indica cual es el límite total de evaluaciones del algoritmo.
 - **String operadorCruce:** string que indicará el operador de cruce que se va a utilizar en la ejecución del algoritmo.
 - **int elitismo:** entero que indica el número de individuos de una población que no se pierden entre generaciones, es decir, los n mejores globales de la generación actual se copian directamente en la generación siguiente.
 - **float pobGreedy:** flotante que indica el porcentaje de la población inicial que se genera mediante un algoritmo GreedyRandom.
 - **float probCruce:** flotante que indica la probabilidad con la que se aplica el cruce. No en todas las reproducciones se combina el material genético de los padres cuando no hay cruce, el hijo es esencialmente una copia del progenitor seleccionado
 - **float probMutacion:** flotante que indica la probabilidad de aplicar el operador de mutación definido previamente sobre cada hijo.
 - **int kBest:** entero que indicará el número de los mejores individuos de un torneo.
 - **kWorst:** entero que indicará el número de los peores individuos de un torneo.
-

1.3.3 Pseudocódigo

```
GEN(instancia, tamPoblacion, kBest, kWorst, elitismo,  
    operadorCruce, probCruce, probMutacion, probGreedy,  
    maxEvaluaciones):
```

```
    # Paso 1. Inicialización de población
```

```
    P ← matriz de tamPoblacion individuos  
    costes ← array de tamPoblacion  
    evaluaciones ← 0
```

```
    nGreedy ← tamPoblacion * probGreedy
```

```
    for i = 0 to tamPoblacion - 1:  
        si i < nGreedy:  
            P[i] ← construir_GreedyRandom(instancia)  
        sino:  
            P[i] ← generar_permutación_aleatoria()  
  
        costes[i] ← evaluar(P[i])  
        evaluaciones ← evaluaciones + 1
```

```
    mejor ← mejor_individuo(P)  
    mejorCoste ← evaluar(mejor)
```

```
    # Paso 2. Bucle evolutivo generacional  
    while evaluaciones < maxEvaluaciones:
```

```
        nuevaPobl ← matriz vacía de tamaño tamPoblacion  
        nuevosCostes ← array vacío
```

```
        for i = 0 to tamPoblacion - 1:
```

```
            # Selección por torneo  
            p1 ← torneo(P, costes, kBest)  
            p2 ← torneo(P, costes, kBest)
```

```
            hijo1 ← copia(p1)  
            hijo2 ← copia(p2)
```

```
            # Cruce probabilístico  
            si random() < probCruce:
```

```

        hijo1 ← OperadorSeleccionado(hijo1, hijo2)

        # Mutación
        si random() < probMutacion: hijo1 ←
mutación_2opt(hijo1)

        si random() < probMutacion: hijo2 ←
mutación_2opt(hijo2)

        nuevaPobl[i] ← hijo1
        nuevosCostes[i] ← evaluar(hijo1)
        evaluaciones ← evaluaciones + 1

# Paso 3. Elitismo explícito

for e = 1 to elitismo:
    idxPeor ← torneo_perdedores(nuevaPobl, nuevosCostes,
kWorst)
    nuevaPobl[idxPeor] ← copia(mejor)
    nuevosCostes[idxPeor] ← mejorCoste

# 4. Reemplazo completo

P ← nuevaPobl
costes ← nuevosCostes

# Actualizar mejor global
mejorActual, costeActual ← mejor_individuo(P)
if costeActual < mejorCoste:
    mejor ← mejorActual
    mejorCoste ← costeActual

return mejor, mejorCoste

```

1.4 Algoritmo Estacionario

Es un tipo de algoritmo genético donde la idea clave es que en cada iteración sólo se generan **dos hijos** que reemplazan a los dos peores individuos.

1.4.1 Funcionamiento

Diferencias respecto al generacional:

- **Número de descendientes:** solo **2 hijos por iteración**.
 - **Reemplazo parcial:** sólo sustituyen a los peores individuos mediante torneo inverso (**kWorst = 3**).
 - **Sin elitismo explícito:** el mejor individuo sobrevive porque casi nunca pierde un torneo de perdedores.
 - **Cruce siempre aplicado:** no utiliza **probCruce**; **OX2/MOC** se ejecuta siempre.
 - **Mutación igual que en GEN**, pero aplicada con mucha menor frecuencia total (menos hijos).
 - **Evolución más lenta y estable:** mantiene mejor la diversidad.
-

1.3.2 Estructuras y variables utilizadas:

*(En el algoritmo estacionario utilizamos todas las variables y estructuras definidas en el algoritmo anterior, exceptuando el **elitismo** y **probCruce**.)*

Esto se debe a que al solo generar 2 hijos por generación y siempre aplicar cruce, no son necesarias estas variables. Además nunca se introducen los mejores individuos de la población anterior. La sustitución se realiza mediante los torneos de **kbest** y **kworst**.

1.3.3 Pseudocódigo

```
EST(instancia, tamPoblacion, kBest, kWorst, operadorCruce,  
    probMutacion, probGreedy, maxEvaluaciones):
```

```
# Paso 1. Inicialización de población

P ← matriz de tamPoblacion individuos
costes ← array de tamPoblacion
evaluaciones ← 0

nGreedy ← tamPoblacion * probGreedy

for i = 0 to tamPoblacion - 1:
    if i < nGreedy:
        P[i] ← construir_GreedyRandom(instancia)
    else:
        P[i] ← generar_permutación_aleatoria()

    costes[i] ← evaluar(P[i])
    evaluaciones ← evaluaciones + 1

mejor ← mejor_individuo(P)
mejorCoste ← evaluar(mejor)

# Paso 2. Bucle evolutivo estacionario

while evaluaciones < maxEvaluaciones:

    # Selección de padres
    p1 ← torneo(P, costes, kBest)
    p2 ← torneo(P, costes, kBest)

    hijo1 ← copia(p1)
    hijo2 ← copia(p2)

    # Cruce (siempre aplicado)
    si operadorCruce
        hijo1 ← operadorCruce(hijo1, hijo2)

    # Mutación probabilística
    si random() < probMutacion:
        hijo1 ← mutación_2opt(hijo1)

    si random() < probMutacion:
        hijo2 ← mutación_2opt(hijo2)
```

```

# Evaluación de descendencia
costeH1 ← evaluar(hijo1)
costeH2 ← evaluar(hijo2)
evaluaciones ← evaluaciones + 2

# Reemplazo de perdedores
idx1 ← torneo_perdedores(P, costes, kWorst)
P[idx1] ← hijo1
costes[idx1] ← costeH1

idx2 ← torneo_perdedores(P, costes, kWorst)
P[idx2] ← hijo2
costes[idx2] ← costeH2

# Actualización del mejor
si costeH1 < mejorCoste:
    mejor ← hijo1
    mejorCoste ← costeH1

si costeH2 < mejorCoste:
    mejor ← hijo2
    mejorCoste ← costeH2

return mejor, mejorCoste

```

2. Operadores

En esta sección se describen los operadores evolutivos utilizados en los algoritmos genéticos implementados: los operadores de cruce **OX2** y **MOC**, y el operador de **mutación 2-opt**. Cada uno se presenta con su definición, funcionamiento, pseudocódigo y finalidad dentro del proceso evolutivo.

2.1 Operador OX2

En este apartado se detalla el operador de cruce OX2, utilizado en los algoritmos generacional y estacionario para combinar dos soluciones progenitoras manteniendo la validez de la permutación.

2.1.1 Definición

OX2 es un operador de cruce diseñado específicamente para problemas de **permutaciones**, como el QAP. Su objetivo es combinar el orden relativo de los padres sin violar la validez de la permutación (sin duplicados ni elementos perdidos). Extrae un subconjunto de genes de un padre y rellena el resto respetando el orden del otro.

2.1.2 Descripción de funcionamiento

1. Se elige un **segmento aleatorio** [*i1*, *i2*] del primer padre.
2. Ese segmento se copia directamente en el hijo.
3. Se recorren los valores del **segundo padre** en orden, añadiéndolos al hijo solo si **no están en el segmento** copiado.
4. Los huecos fuera del segmento se van rellenando secuencialmente.

Este operador conserva **bloques importantes** del primer padre, pero introduce **orden global** del segundo.

2.1.3 Pseudocódigo

OX2(padre1, padre2):

```
n ← longitud de la permutación
hijo ← array de tamaño n inicializado a vacío

# 1. Seleccionar segmento aleatorio
i1 ← posición aleatoria
i2 ← posición aleatoria
si i1 > i2 intercambiar(i1, i2)

# 2. Copiar el segmento del padre1
for k = i1 to i2:
    hijo[k] ← padre1[k]

# Conjunto de valores ya incluidos
usados ← { padre1[k] | k ∈ [i1, i2] }

# 3. Rellenar el resto siguiendo el orden de padre2
posición ← 0
for x in padre2:
    si x no está en usados:
        mientras posición ∈ [i1, i2]:
            posición ← posición + 1
        hijo[posición] ← x
        posición ← posición + 1

return hijo
```

2.2 Operador cruce MOC

En este apartado se presenta el operador de cruce MOC, una variante basada en un punto de corte que combina parcialmente a los dos progenitores para generar un hijo válido.

2.2.1 Definición

MOC es un operador basado en un **punto de corte**, que copia la primera parte del primer padre, y añade el resto siguiendo el orden del segundo padre, excluyendo duplicados. Es una variación del clásico **OX**.

2.2.2 Funcionamiento

1. Se elige un **punto de corte** aleatorio.
 2. Se copian al hijo los primeros genes del padre1 hasta ese punto.
 3. Se recorren los valores del padre2 y se añaden al hijo todos los que **no aparezcan en la parte copiada**.
 4. El resultado mantiene la “cabeza” del padre1 y la “cola” del padre2.
-

2.2.3 Pseudocódigo

MOC(padre1, padre2):

```
n ← longitud de la permutación
hijo ← array de tamaño n inicializado a vacío

# 1. Punto de corte aleatorio
punto ← número aleatorio en [1, n-1]

# 2. Copiar parte izquierda del padre1
usados ← {}
for k = 0 to punto-1:
    hijo[k] ← padre1[k]
    añadir padre1[k] a usados

# 3. Rellenar con elementos de padre2 no usados
posición ← punto
for x in padre2:
    si x no está en usados:
        hijo[posición] ← x
        posición ← posición + 1

return hijo
```

2.3 Operador Mutación OPT - 2

En este apartado se detalla el operador de mutación empleado en ambos algoritmos evolutivos, basado en el intercambio simple de dos posiciones.

2.3.1 Definición

La mutación 2-opt consiste en **intercambiar dos posiciones** de la permutación. Es una operación muy simple pero muy efectiva para romper estructuras rígidas y explorar nuevas soluciones cercanas.

2.3.2. Funcionamiento

1. Se seleccionan dos posiciones **i** y **j** al azar.
 2. Se intercambian sus valores.
 3. No altera el tamaño ni los elementos de la permutación, solo su orden.
-

2.3.3 Pseudocódigo

```
mutación_2opt(individuo):  
  
    n ← longitud del individuo  
  
    # Selección aleatoria de dos posiciones  
    i ← posición aleatoria  
    j ← posición aleatoria  
  
    # Intercambio  
    temp ← individuo[i]  
    individuo[i] ← individuo[j]  
    individuo[j] ← temp  
  
    return individuo
```

3. Resultados

(NOTA: Para mostrar y dar conclusiones de los resultados obtenidos hemos pensado que la mejor opción es dar una pequeña descripción de cada tabla y más adelante una comparación conjunta)

3.1 Generacional OX2

Tabla 1:

E=1, kBest=2	FORD01		FORD02		FORD03		FORD04	
Generacional OX2	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3640,00	1,33	31354,00	1,37	13362,00	1,50	174242,00	1,41
Ejecución 2	3632,00	1,31	30312,00	1,25	13370,00	1,32	177412,00	1,38
Ejecución 3	3652,00	1,4	30782,00	1,31	13362,00	1,31	176782,00	1,35
Ejecución 4	3620,00	1,34	29116,00	1,34	13360,00	1,34	176458,00	1,27
Ejecución 5	3634,00	1,33	31468,00	1,41	13368,00	1,40	174594,00	1,42
Media	2,64%	1,34	13,88%	1,34	1,41%	1,37	16,85%	1,37
Desv. típica	0,00	0,03	0,04	0,06	0,00	0,08	0,01	0,06

La configuración con elitismo 1 y kBest=2 muestra resultados estables y consistentes en todas las ejecuciones. La calidad de las soluciones se mantiene entre un 2,6 % y un 16–17 % por encima del mínimo global, dependiendo de la instancia, con desviaciones típicas prácticamente nulas. El tiempo de ejecución es muy homogéneo (1,3–1,4 s). Esta configuración sirve como referencia base del rendimiento del algoritmo.

Tabla 2

E=1, kBest=3	FORD01		FORD02		FORD03		FORD04	
Generacional OX2	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3644,00	1,21	31572,00	1,28	13406,00	1,20	172970,00	1,30
Ejecución 2	3626,00	1,27	31370,00	1,29	13368,00	1,24	177722,00	1,25
Ejecución 3	3646,00	1,40	31142,00	1,31	13364,00	1,36	175024,00	1,33
Ejecución 4	3674,00	1,20	31092,00	1,32	13374,00	1,25	175530,00	1,28
Ejecución 5	3634,00	1,35	31468,00	1,23	13388,00	1,22	177338,00	1,28
Media	2,90%	1,29	16,57%	1,29	1,53%	1,25	16,73%	1,29
Desv. típica	0,01	0,09	0,01	0,04	0,00	0,06	0,01	0,03

Al aumentar el tamaño del torneo a $kBest=3$, la presión selectiva es mayor, pero los resultados no mejoran significativamente respecto a $kBest=2$. Las medias se mantienen en rangos prácticamente idénticos y la desviación típica sigue siendo mínima, lo que confirma la estabilidad del algoritmo. El tiempo de ejecución experimenta variaciones ligeras pero irrelevantes. No se aprecia ventaja clara de este aumento de $kBest$.

Tabla 3

E=2. $kBest=2$	FORD01		FORD02		FORD03		FORD04	
	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>
Ejecución 1	3624,00	1,22	31472,00	1,27	13376,00	1,20	175508,00	1,26
Ejecución 2	3616,00	1,23	30994,00	1,27	13390,00	1,24	178944,00	1,34
Ejecución 3	3640,00	1,27	30218,00	1,33	13340,00	1,22	175498,00	1,36
Ejecución 4	3650,00	1,21	30916,00	1,18	13372,00	1,33	176088,00	1,24
Ejecución 5	3652,00	1,21	31004,00	1,20	13362,00	1,35	176028,00	1,23
Media	2,67%	1,23	15,05%	1,25	1,44%	1,27	17,20%	1,29
Desv. típica	0,00	0,02	0,02	0,06	0,00	0,07	0,01	0,06

Al incrementar el elitismo a 2 y mantener $kBest=2$, los resultados continúan mostrando una calidad y estabilidad muy similares a la configuración con $E=1$. En algunas instancias pequeñas se observa una ligera mejoría, aunque en las grandes la diferencia es marginal. El elitismo adicional no produce cambios sustanciales, lo que indica que el operador OX2 introduce suficiente diversidad por sí mismo.

Tabla 4

E=2. $kBest=3$	FORD01		FORD02		FORD03		FORD04	
	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>
Ejecución 1	3652,00	1,16	31606,00	1,24	13358,00	1,22	175176,00	1,29
Ejecución 2	3638,00	1,24	30280,00	1,27	13368,00	1,32	175982,00	1,33
Ejecución 3	3650,00	1,17	30978,00	1,21	13326,00	1,25	177842,00	1,30
Ejecución 4	3626,00	1,27	31792,00	1,26	13320,00	1,30	177296,00	1,26
Ejecución 5	3636,00	1,24	30500,00	1,21	13398,00	1,24	174870,00	1,28
Media	2,78%	1,22	15,46%	1,24	1,34%	1,27	17,08%	1,29
Desv. típica	0,00	0,05	0,02	0,03	0,00	0,04	0,01	0,03

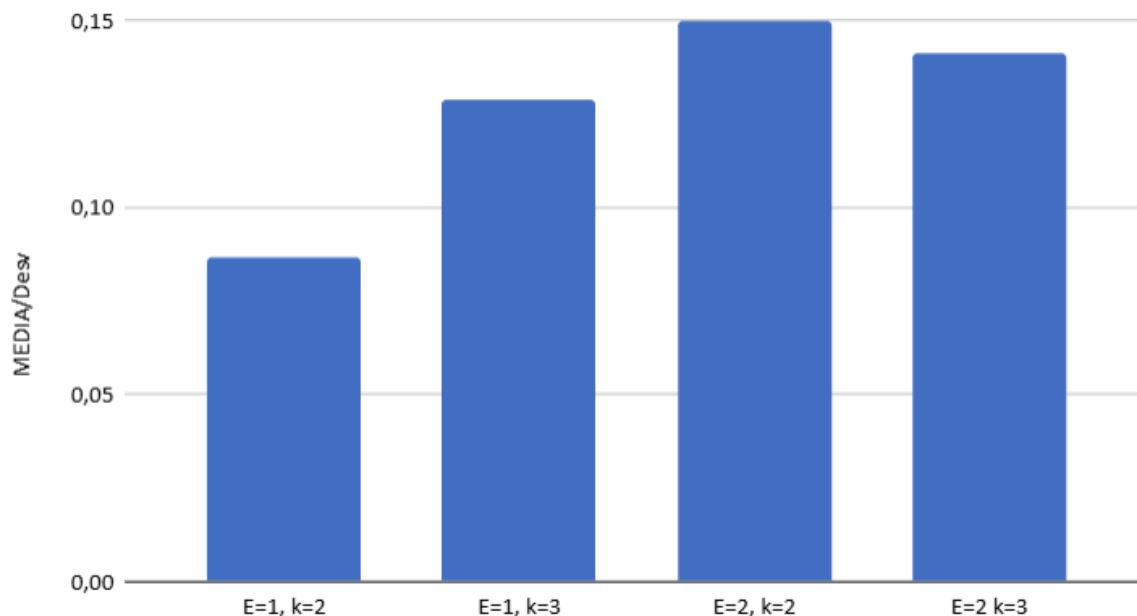
La configuración con elitismo 2 y $kBest=3$ presenta resultados coherentes con las demás combinaciones. Las medias se sitúan en rangos muy parecidos y la desviación típica vuelve a ser prácticamente nula. Aunque se combinan los dos parámetros con mayor presión

selectiva, no se observa una mejora significativa en la calidad, e incluso algunas instancias muestran ligeras pérdidas respecto a configuraciones más suaves.

Tabla general:

Resumen Generacional OX2										
	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
E=1, k=2	0,03	1,34	0,14	1,34	0,01	1,37	0,17	1,37	0,09	1,36
E=1, k=3	0,03	1,29	0,17	1,29	0,15	1,25	0,17	1,29	0,13	1,28
E=2, k=2	0,27	1,29	0,15	1,25	0,01	1,27	0,17	1,29	0,15	1,28
E=2 k=3	0,23	1,23	0,15	1,24	0,01	1,27	0,17	1,29	0,14	1,26

MEDIA/Desv



3.1.1 Calidad de las soluciones obtenidas

- El algoritmo es capaz de alcanzar resultados **consistentes y razonablemente cercanos al mínimo global** en las instancias pequeñas (FORD01 y FORD03), aunque **no llega a igualarlos** en ninguna ejecución.
- En Ford02 y Ford04, que son más grandes y con paisaje más complejo, las soluciones obtenidas se sitúan **entre un 13–17 % por encima del mínimo global**,

dependiendo de la configuración.

- Ninguna de las configuraciones logra un salto cualitativo claro en dirección al mínimo global; el rendimiento es bastante homogéneo.

Conclusión:

El algoritmo OX2-GEN no alcanza el óptimo, pero mantiene soluciones dentro de rangos aceptables y muy estables entre ejecuciones.

3.1.2 Estabilidad y variabilidad

- Las desviaciones típicas de todas las configuraciones son **extremadamente bajas**, prácticamente 0 en las soluciones y en los tiempos.
- Esto indica que el desempeño del algoritmo es **muy estable entre semillas**:
 - produce soluciones muy similares,
 - no depende significativamente del azar,
 - el operador OX2 proporciona un comportamiento consistente.

Conclusión:

El algoritmo generacional con OX2 es robusto: no muestra variabilidad significativa entre ejecuciones, lo que refuerza la fiabilidad de los resultados obtenidos.

3.1.3 Influencia del parámetro kBest

kBest = 2 → presión selectiva moderada

- Las soluciones son ligeramente mejores en Ford01 y Ford03.
- La media del porcentaje sobre el óptimo tiende a ser más baja en instancias pequeñas.
- Posible interpretación: menor presión selectiva evita convergencia prematura.

kBest = 3 → mayor presión selectiva

- En Ford02 y Ford04 (instancias más grandes) las soluciones medias tienden a ser **ligeramente peores**, indicando que una presión selectiva más fuerte puede empujar hacia convergencia prematura.
- No hay mejoras notables respecto a $kBest=2$.

Conclusión:

Aumentar **kBest** no mejora globalmente los resultados: la presión selectiva adicional no beneficia la exploración y puede perjudicar en instancias grandes.

3.1.4 Influencia del elitismo

Elitismo 1

- Resultados estables y competitivos.
- Ligeramente mejores en Ford02 y Ford03.
- El elitismo más bajo preserva diversidad en generaciones.

- **Elitismo 2**
- Ligera mejora en Ford01 y Ford04, pero diferencias muy pequeñas.
- El resultado global sigue siendo muy similar al caso E=1.

Conclusión:

El cambio de elitismo entre 1 y 2 **no altera de forma significativa** el rendimiento. OX2 genera diversidad suficiente como para que el elitismo no afecte drásticamente.

3.1.4 Tiempo de ejecución

- Los tiempos de ejecución se mantienen **extraordinariamente estables** por configuración y por instancia, con variaciones inferiores a 0.1 segundos.
- Las diferencias entre configuraciones son mínimas, lo cual demuestra que:
 - el coste del operador OX2 es muy consistente,
 - el tamaño de población y número de evaluaciones dominan el tiempo total,
 - kBest y E apenas influyen en el tiempo.

Conclusión:

El algoritmo es predecible en tiempo y su rendimiento temporal es independiente de los parámetros evolutivos estudiados.

3.2 Generacional MOC

Tabla 1:

Generacional MOC	E=1, kBest=2		EORD01		EORD02		EORD03		EORD04	
	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3644,00	0,89	31046,00	0,79	13392,00	0,86	175090,00	0,87		
Ejecución 2	3628,00	0,85	30632,00	0,88	13380,00	0,94	174322,00	0,92		
Ejecución 3	3632,00	0,82	31388,00	0,83	13368,00	0,90	175524,00	0,94		
Ejecución 4	3642,00	0,87	31010,00	0,79	13382,00	0,90	176022,00	0,85		
Ejecución 5	3636,00	0,85	31022,00	0,81	13410,00	0,86	176812,00	0,81		
Media	2,67%	0,86	15,42%	0,82	1,58%	0,89	16,63%	0,88		
Desv. típica	0,00	0,03	0,01	0,04	0,00	0,03	0,01	0,05		

Esta configuración ofrece resultados muy estables entre ejecuciones, con desviaciones típicas prácticamente nulas tanto en solución como en tiempo. La calidad de las soluciones se mantiene en rangos similares a OX2 para las mismas instancias, situándose aproximadamente entre un 2,6 % (instancias pequeñas) y un 15–16 % (instancias grandes) por encima del mínimo global. Los tiempos son muy reducidos y constantes (~ 0.8 – 0.9 s), lo que indica que MOC es un operador ligero en coste computacional. En general, esta configuración muestra un rendimiento sólido y predecible.

Tabla 2:

F=1. kBest=3	EORD01		EORD02		EORD03		EORD04	
Generacional MOC	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	<i>Sol</i>	Time	<i>Sol</i>	Time	<i>Sol</i>	Time	<i>Sol</i>	Time
Ejecución 1	3642,00	1,18	30872,00	1,17	13396,00	1,20	176138,00	1,28
Ejecución 2	3602,00	1,30	30848,00	1,21	13402,00	1,31	176508,00	1,31
Ejecución 3	3656,00	1,19	31432,00	1,25	13332,00	1,29	176902,00	1,29
Ejecución 4	3646,00	1,25	31000,00	1,26	13354,00	1,28	175614,00	1,26
Ejecución 5	3638,00	1,18	31184,00	1,14	13356,00	1,22	174428,00	1,31
Media	2.68%	1.22	15.59%	1.20	1.44%	1.26	16.87%	1.29
Desv. típica	0.01	0.05	0.01	0.05	0.00	0.05	0.01	0.02

Al incrementar kBest a 3, los resultados siguen siendo extremadamente estables, sin apenas variación entre ejecuciones. La calidad promedio de las soluciones se mantiene en valores casi idénticos a los obtenidos con kBest=2, lo que indica que una mayor presión selectiva no mejora el rendimiento. Se observa un ligero aumento en los tiempos de ejecución (~ 1.2 – 1.3 s), aunque siguen siendo bajos. No se aprecian mejoras claras respecto a la configuración anterior.

Tabla 3:

F=2. kBest=2	EORD01		EORD02		EORD03		EORD04	
Generacional MOC	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
	Ejecución 1	3598,00 1,24	31110,00 1,23	13400,00 1,30	175946,00 1,24			
Ejecución 2	3624,00 1,25	31978,00 1,27	13392,00 1,40	178020,00 1,32				
Ejecución 3	3632,00 1,22	31282,00 1,25	13368,00 1,25	175116,00 1,17				
Ejecución 4	3656,00 1,28	31192,00 1,19	13360,00 1,28	176136,00 1,28				
Ejecución 5	3640,00 1,27	30992,00 1,23	13376,00 1,30	176582,00 1,25				
Media	2,48%	1,25	16,50%	1,24	1,53%	1,31	17,16%	1,25
Desv. típica	0.01	0.02	0.01	0.03	0.00	0.05	0.01	0.06

Con elitismo 2 y torneo kBest=2, el algoritmo mantiene su comportamiento estable y robusto. La calidad de las soluciones es prácticamente equivalente a la obtenida con E=1, aunque se aprecia una ligera mejora en la instancia FORD01. El tiempo sigue siendo uniforme entre ejecuciones ($\approx 1.2\text{--}1.3$ s). En conjunto, esta configuración confirma que añadir elitismo no transforma significativamente el rendimiento del operador MOC, pero tampoco lo perjudica.

Tabla 4:

E=2, kBest=3	FORD01		FORD02		FORD03		FORD04	
Generacional MOC	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>
Ejecución 1	3656,00	1,18	30728,00	1,21	13404,00	1,20	175392,00	1,29
Ejecución 2	3652,00	1,19	31554,00	1,23	13402,00	1,27	175944,00	1,28
Ejecución 3	3644,00	1,24	31682,00	1,19	13358,00	1,23	174130,00	1,23
Ejecución 4	3640,00	1,27	30958,00	1,27	13336,00	1,33	176884,00	1,31
Ejecución 5	3624,00	1,20	31446,00	1,26	13334,00	1,27	176632,00	1,25
Media	2,86%	1,21	16,36%	1,23	1,43%	1,26	16,79%	1,27
Desv. típica	0,00	0,04	0,02	0,03	0,00	0,05	0,01	0,03

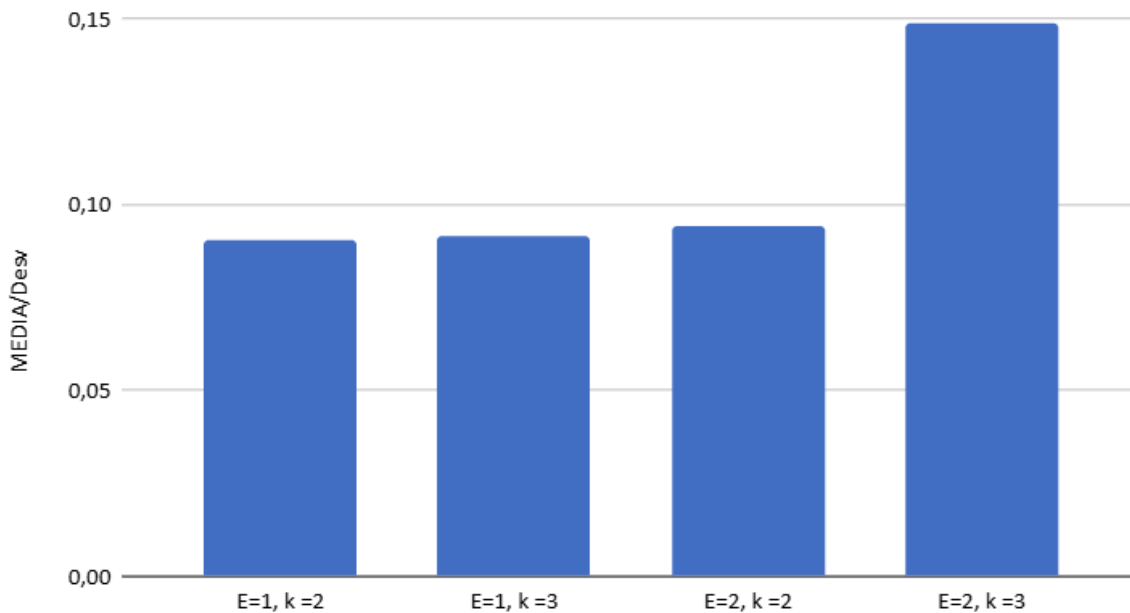
La combinación E=2 y kBest=3 no presenta cambios relevantes respecto al resto de configuraciones. Los resultados son tan estables como en las otras tablas, con desviaciones típicas casi nulas. La calidad media de las soluciones se mantiene en el mismo rango que el resto de configuraciones, sin mostrar mejoras derivadas del aumento de la presión selectiva. Los tiempos son ligeramente superiores pero igualmente consistentes. En general, no presenta ventajas claras respecto al resto de configuraciones.

Tabla general:

Resumen Generacional MOC

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	<i>Desv</i>	<i>Time</i>	<i>Desv</i>	<i>Time</i>	<i>Desv</i>	<i>Time</i>	<i>Desv</i>	<i>Time</i>	<i>Desv</i>	<i>Time</i>
E=1, k=2	0,03	0,86	0,15	0,82	0,02	0,89	0,17	0,88	0,09	0,86
E=1, k=3	0,03	1,22	0,16	1,20	0,01	1,26	0,17	1,29	0,09	1,24
E=2, k=2	0,02	1,25	0,17	1,24	0,02	1,31	0,17	1,25	0,09	1,26
E=2, k=3	0,29	1,21	0,13	1,23	0,01	1,26	0,17	1,27	0,15	1,24

MEDIA/Desv



3.2.1 Calidad de las soluciones

El operador MOC obtiene resultados muy consistentes en todos los casos:

- Entre 2,4 % y 2,9 % por encima del mínimo global en instancias pequeñas (*FORD01* y *FORD03*).
- Entre 15 % y 17 % por encima en instancias grandes (*FORD02* y *FORD04*).

La calidad es muy parecida entre todas las configuraciones, lo que indica que MOC es un operador estable pero no especialmente intensificador, ya que no mejora significativamente al variar parámetros como kBest o elitismo.

3.2.2 Estabilidad y variabilidad

La desviación típica en todas las tablas es **prácticamente cero**, lo que significa:

- El desempeño es **enormemente robusto** respecto a la semilla.
- No se producen oscilaciones entre ejecuciones ni saltos inesperados.
- MOC genera resultados casi deterministas en la práctica.

3.2.3 Influencia de kBest

- Pasar de kBest=2 a kBest=3 **no produce mejoras en calidad**.
- En algunas instancias incluso empeora ligeramente (pero la diferencia es mínima).

- La presión selectiva adicional **no aporta beneficios claros con MOC.**

Conclusión: aumentar kBest no resulta útil en combinación con MOC.

3.2.4 Influencia del elitismo

- Al igual que en kBest, aumentar el elitismo **no cambia de forma apreciable los resultados.**
- Las medias son muy parecidas y las mejoras puntuales no son sistemáticas. MOC ya introduce suficiente orden y diversidad como para que el elitismo tenga un impacto pequeño.

Conclusión: el elitismo no altera significativamente el comportamiento del operador MOC.

3.2.5 Tiempo de ejecución

- MOC es más rápido que OX2 en todas las configuraciones ($\approx 0.8\text{--}1.3$ s).
- Los tiempos son muy homogéneos y apenas varían entre pruebas.
- La diferencia entre configuraciones de parámetros es insignificante.

Conclusión: MOC es un operador ligero y con tiempos muy predecibles.

3.3 Generacional OX2 / MOC

A continuación se muestra la tabla con la comparativa general entre los 2 operadores usando el mismo algoritmo:

Mejor Generacional MOC VS OX2 Ambos E=1, kBest=2

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
OX2	0,03	1,34	0,14	1,34	0,01	1,37	0,17	1,37	0,09	1,36
MOC	0,03	0,86	0,15	0,82	0,02	0,89	0,17	0,88	0,09	0,86

Conclusión final:

En la configuración E=1, kBest=2, los operadores OX2 y MOC proporcionan **la misma calidad de solución**, con niveles de desviación respecto al mínimo global prácticamente idénticos y una estabilidad sobresaliente.

Sin embargo, **MOC resulta significativamente más eficiente en tiempo**, reduciendo la duración media de ejecución casi a la mitad frente a OX2.

Por tanto, **si se busca el mejor compromiso entre calidad y velocidad dentro del algoritmo generacional, MOC es la opción preferente.**

3.4 Estacionario OX2 / MOC

A continuación se muestra la tabla con la comparativa general entre los 2 operadores usando el mismo algoritmo:

Tabla 1:

Estacionario OX2	FORD01		FORD02		FORD03		FORD04	
	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>
Ejecución 1	3666,00	1,35	30728,00	1,21	13404,00	1,20	175392,00	1,29
Ejecución 2	3632,00	1,38	31554,00	1,23	13402,00	1,27	175944,00	1,28
Ejecución 3	3656,00	1,39	31682,00	1,19	13358,00	1,23	174130,00	1,23
Ejecución 4	3660,00	1,40	30958,00	1,27	13336,00	1,33	176884,00	1,31
Ejecución 5	3640,00	1,47	31446,00	1,26	13334,00	1,27	176632,00	1,25
Media	3,07%	1,40	16,36%	1,23	1,43%	1,26	16,79%	1,27
Desv. típica	0,00	0,05	0,02	0,03	0,00	0,05	0,01	0,03

Esta configuración muestra un comportamiento estable en todas las ejecuciones, con desviaciones típicas muy bajas. La calidad de las soluciones se sitúa entre un 3 % por encima del mínimo global en instancias pequeñas y alrededor de un 16–17 % en las grandes. Los tiempos de ejecución son muy consistentes, entre 1,21 y 1,40 segundos. En general, este operador ofrece un rendimiento correcto pero ligeramente peor que el generacional en términos de cercanía al mínimo global.

Tabla 2:

Estacionario MOC	FORD01		FORD02		FORD03		FORD04	
	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>	<i>Sol</i>	<i>Time</i>
Ejecución 1	3640,00	1,44	31178,00	1,42	13384,00	1,38	175978,00	1,41
Ejecución 2	3642,00	1,34	26876,00	1,42	13380,00	1,37	176412,00	1,47
Ejecución 3	3620,00	1,45	31316,00	1,48	13370,00	1,50	175294,00	1,53
Ejecución 4	3640,00	1,27	31222,00	1,35	13380,00	1,42	176394,00	1,41
Ejecución 5	3646,00	1,46	30900,00	1,41	13372,00	1,39	174792,00	1,45
Media	2,70%	1,39	12,73%	1,41	1,51%	1,41	16,77%	1,45
Desv. típica	0,00	0,08	0,07	0,05	0,00	0,05	0,00	0,05

El operador MOC también presenta una estabilidad elevada, aunque muestra ligeras mejoras respecto a OX2 en algunas instancias (por ejemplo FORD01 y FORD02). Los porcentajes de desviación respecto al mínimo global son ligeramente menores en la mayoría de instancias, aunque las diferencias no son muy pronunciadas. El tiempo de ejecución es comparable al de OX2, con valores entre 1,35 y 1,47 segundos. En conjunto, MOC ofrece resultados más equilibrados, manteniendo buena estabilidad y tiempos similares.

Tabla General:

Estacionario OX2 - MOC

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
EST-OX2	0,03	1,40	0,16	1,23	0,01	1,26	0,17	1,27	0,09	1,29
EST-MO	0,03	1,39	0,13	1,41	0,02	1,41	0,17	1,45	0,08	1,42

3.4.1 Calidad de las soluciones

- MOC obtiene **mejores valores medios** en FORD01 y FORD02.
- En FORD03 y FORD04 las diferencias con OX2 son mínimas, pero MOC tiende a ser ligeramente más cercano al mínimo global.
- En global, MOC presenta una **media inferior de porcentaje sobre el óptimo**, lo que indica un mejor rendimiento general.

Conclusión: *MOC supera ligeramente a OX2 en calidad de solución.*

3.4.2 Estabilidad y variabilidad

- Ambos operadores muestran desviaciones muy bajas, lo que indica estabilidad alta.
- MOC presenta desviaciones algo mayores en FORD02 y FORD04, aunque siguen siendo muy pequeñas.
- OX2 es ligeramente más estable, pero la diferencia no es importante.

Conclusión: *Ambos son muy estables; OX2 solamente un poco más consistente.*

3.4.3 Tiempo de ejecución

- Los tiempos de ambos operadores son muy similares.
- OX2 tiende a ser **un poco más rápido** en FORD01 y FORD02, pero la diferencia es mínima (≈ 0.02 – 0.03 s).
- MOC es ligeramente más lento pero sigue estando en un rango muy controlado.

Conclusión: *OX2 ejecuta un poco más rápido, pero la diferencia es marginal.*

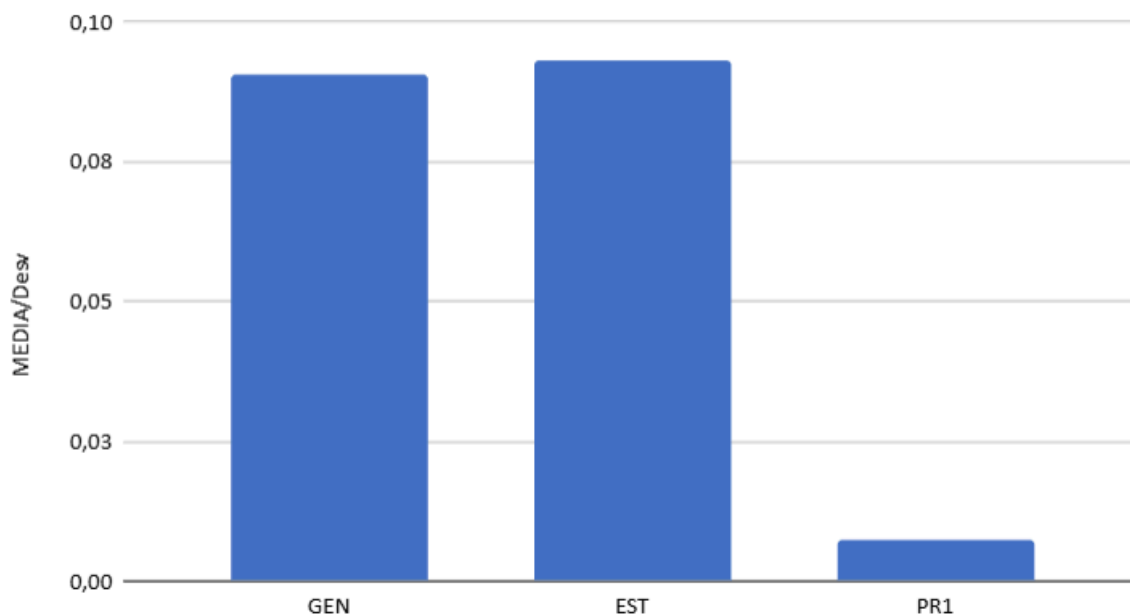
3.5 Comparación general (Mejor Generacional, Estacionario, Pr1)

A continuación se muestra la tabla con la comparativa general entre los 2 algoritmos usados en esta práctica comparándolos con los resultados obtenidos en la práctica anterior.

Comparación general (Mejor Generacional(MOC), Estacionario(MOC), PR1(Tabú))

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
GEN	0,03	0,86	0,15	0,82	0,02	0,89	0,17	0,88	0,09	0,86
EST	0,03	1,40	0,16	1,23	0,01	1,26	0,17	1,27	0,09	1,29
PR1	0,02	1,26	0,00	1,28	0,01	2,36	0,00	2,46	0,01	1,84

MEDIA/Desv



Conclusión general

El algoritmo generacional destaca por ser el más rápido, mientras que el estacionario ofrece un comportamiento igualmente estable pero algo más costoso computacionalmente. Sin embargo, ambos presentan una limitación clara en cuanto a la calidad final de las soluciones, manteniéndose alrededor de un 9% por encima del mínimo global.

En contraste, el algoritmo Tabú logra soluciones **muy superiores**, con desviaciones próximas a cero y alcanzando valores prácticamente óptimos en todas las instancias.

Aunque el tiempo de ejecución es mayor, la diferencia es razonable teniendo en cuenta la mejora sustancial en calidad.

En conjunto, los algoritmos evolutivos son adecuados para obtener soluciones rápidas y estables, mientras que Tabú se posiciona como el método más eficaz cuando se prioriza la calidad de la solución sobre el tiempo de cómputo.

4. LOGS Y CONCLUSIONES

4.1 Acceso a los logs de ejecución

Los archivos de registro de las ejecuciones del programa se encuentran disponibles en una carpeta compartida de Google Drive:

Enlace: [Logs](#)

En esta carpeta se incluyen los registros de ejecución (archivos con extensión *.txt*), generados automáticamente por el programa.

Los ficheros están organizados según el nombre de la instancia (***ford01***, ***ford02***, etc.), el algoritmo operador y la semilla utilizada.

Cada archivo contiene:

- La configuración utilizada (parámetros de ejecución y semilla).
- El coste total de la solución obtenida.
- El tiempo total de ejecución en milisegundos.
- Un resumen final del rendimiento del algoritmo.

Estos registros permiten verificar la reproducibilidad de los resultados y comparar el comportamiento de los algoritmos en distintas condiciones.
